

# OCA BEU — Low-Level Design

---

## SMC Bus Error Unit — Low-Level Design

1. Module structure
2. TLM-2.0 interface
3. Register decode
4. Error injection (back door)
5. Processes and events
6. Interrupt aggregation
7. Debug / introspection
8. Configuration (CCI)
9. Known limitations

# SMC Bus Error Unit — Low-Level Design

Implementation notes for the SystemC/TLM-2.0 LT model in `smc/peripherals/beu` (`include/beu.h`, `src/beu.cpp`). Read `01_BEU_Specification.md` first for externally-observable behaviour.

## 1. Module structure

`smc::beu` is a single `SC_MODULE` with:

- one TLM-2.0 target socket (`reg_socket`, 64-bit AXI4-Lite-style),
- an active-low reset input (`rst_n_i`),
- two boolean interrupt outputs (`irq_local_o`, `irq_plic_o`),
- one CCI parameter (`access_delay_ns`).

It reuses the shared, framework-free register helpers in `common/include`:

- `regmodel::Register64` (`reg_access.h`) — a register with a fixed read/write-mask + reset contract, used for the plain RW registers `ENABLE`, `PLIC_ENABLE`, `LOCAL_ENABLE`.
- `regmodel::RegisterMap64` (`reg_map.h`) — an offset → register dispatch table owning those three registers.

`CAUSE`, `PHYS_ADDR`, and `ACCRUED_ENABLE` carry model-side behaviour (HW-set, first-error latch, sticky accrue) and are kept as plain members handled in the `reg_read` / `reg_write` switch.

## 2. TLM-2.0 interface

`b_transport` is loosely-timed and **never calls** `wait()`. It:

1. rejects non-read/write commands → `TLM_COMMAND_ERROR_RESPONSE`;
2. requires `data_length == 8` → else `TLM_BURST_ERROR_RESPONSE`;
3. requires `addr < 0x1000` and 8-byte alignment → else `TLM_ADDRESS_ERROR_RESPONSE`;
4. dispatches to `reg_read` / `reg_write`; an in-window offset that decodes to no register also returns `TLM_ADDRESS_ERROR_RESPONSE`;
5. adds `access_delay_ns` to the annotated `delay` and sets `dmi_allowed = false`.

`transport_dbg` provides the same decode with **no** timing or side effects (it reads through `dbg_reg`), returning the number of bytes transferred (8) or 0.

Temporal decoupling (quantum keeper) is the initiator's responsibility; the test bench driver owns a `tlm_utils::tlm_quantumkeeper`.

### 3. Register decode

```
reg_read(off):
    CAUSE / PHYS_ADDR / ACCRUED_ENABLE -> return model member
    else                               -> regmap_.read(off)    (ENABLE/PLIC/LOCAL)
    miss                               -> false    (=> ADDRESS_ERROR)

reg_write(off, data):
    CAUSE          -> cause_ = data & 0x7;                recompute
    PHYS_ADDR      -> ignored (read-only to SW)
    ACCRUED_ENABLE -> accrued_ = data & VALID_MASK;        recompute    (SW ack/clear)
    ENABLE/PLIC/LOCAL -> regmap_.write(off,data);          recompute
    miss           -> false    (=> ADDRESS_ERROR)
```

The Register64 write-mask enforces `VALID_MASK` (0xE6) on `ENABLE`, `PLIC_ENABLE`, `LOCAL_ENABLE`, so reserved bits never store.

### 4. Error injection (back door)

`inject_error(beu_src s, uint64_t phys_addr)` models one hardware error pulse:

```
bit   = static_cast<unsigned>(s);           // 1,2,5,6,7
mask  = 1u << bit;
accrued_ |= (mask & VALID_MASK);           // raw sticky status, always
if ((enable_.read() & mask) && cause_ == 0) {
    cause_ = bit;                           // first enabled error only
    phys_addr_ = phys_addr & PHYS_MASK;     // low 56 bits
}
schedule_recompute();
```

Because a single call injects one source, the RTL's multi-source priority mux collapses to “record if `CAUSE==0`”. The priority ordering itself is still observable when several sources are injected back-to-back before the first is cleared (the first-arriving enabled source wins, matching “record while `CAUSE==0`”).

### 5. Processes and events

Two `SC_METHOD`s, both `dont_initialize()`:

- `reset_proc` — sensitive to `rst_n_i`; on the low level restores reset values and schedules a recompute.
- `recompute_method` — sensitive to `recompute_event_`; the **sole driver** of the two interrupt outputs. It recomputes `irq_local/irq_plic` from the accrued status AND-ed with the two masks and writes the signals only when they change (output caching suppresses redundant `sc_signal` writes).

Every state-changing path (`reg_write`, `inject_error`, `reset_proc`) calls `schedule_recompute()`, which does `recompute_event_.notify(SC_ZERO_TIME)`.

## 6. Interrupt aggregation

---

```
irq_local_active() = (accrued_ & local_enable_.read() & VALID_MASK) != 0;  
irq_plic_active()  = (accrued_ & plic_enable_.read() & VALID_MASK) != 0;
```

Level-sensitive: the lines follow the sticky accrued status and de-assert only when software clears the accrued bits (write `ACCRUED_ENABLE = 0`).

## 7. Debug / introspection

---

- `dbg_reg(off)` — side-effect-free peek used by `transport_dbg` and tests.
- `dump_state(os)` — prints CCI parameters, all register values, and the two interrupt states.

## 8. Configuration (CCI)

---

`access_delay_ns (cci_param<double>, default 2.0, mutable)` annotates each `b_transport` access. The constructor logs the resolved value and its provenance (`[preset]` / `[default]`) via `SIM_LOG_INFO`.

## 9. Known limitations

---

- Error sources are injected via the back door, not driven as ports (no cache in the model). Wiring real error-report ports is a straightforward extension.
- No cycle-level timing; a single scalar delay approximates AXI4-Lite latency.
- `ieu_error` (ICache uncorrectable, cause 3) and cause 4 are unused by the SMC configuration and are not produced by the model.